

ADP470S

Applications Development Practice 4

TERM 1 COMPREHENSIVE STUDY NOTES

Topics Covered:

- Algorithms & Data Structures – Introduction
 - Big O Notation & Algorithm Complexity
- Recursion vs Iteration (Factorial & Fibonacci)
- Searching Algorithms (Linear & Binary Search)
- Bubble Sort – Best, Worst & Average Case
 - 2D & Multidimensional Arrays
 - Introduction to Linked Lists (Single)
- Doubly Linked List & Circular Linked List

1. Algorithms & Data Structures – Introduction

1.1 What is an Algorithm?

An algorithm is a finite, well-defined sequence of steps or instructions used to solve a problem or accomplish a task. It must be correct, efficient, and terminate.

Five Essential Properties of an Algorithm

1. Input -- Zero or more inputs are externally supplied
2. Output -- At least one output is produced
3. Definiteness -- Each step must be precisely defined (no ambiguity)
4. Finiteness -- Must terminate after a finite number of steps
5. Effectiveness -- Every step must be basic enough to be carried out

1.2 What are Data Structures?

A data structure is a way of organising, storing, and managing data in a computer so that it can be accessed and modified efficiently. Different problems need different structures.

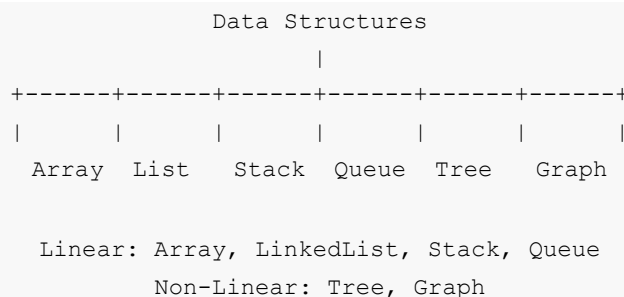


Figure 1.1 – Classification of Data Structures

Data Structure	Access	Search	Insertion	Deletion
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$
BST (average)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

1.3 Why Algorithm Efficiency Matters

Two algorithms can solve the same problem but differ vastly in speed and memory use. For large inputs, an inefficient algorithm can take years while an efficient one finishes in seconds.

$n = 1,000,000$ operations:

$O(1)$	-->	1 operation	(instant)
$O(\log n)$	-->	~20 operations	(very fast)
$O(n)$	-->	1,000,000 ops	(fast)
$O(n \log n)$	-->	~20,000,000	(acceptable)
$O(n^2)$	-->	10^{12} ops	(very slow)
$O(2^n)$	-->	impossible	(never finishes)

Figure 1.2 – Growth rate comparison for $n = 1,000,000$

2. Big O Notation & Algorithm Complexity

2.1 What is Big O Notation?

Big O notation describes the upper bound of an algorithm's running time or space usage as the input size n grows. It expresses the WORST CASE behaviour, ignoring constants and lower-order terms.

Big O Definition

$f(n) = O(g(n))$ means f grows no faster than g as $n \rightarrow \infty$

We drop constants: $3n + 5 = O(n)$

We drop lower terms: $n^2 + n = O(n^2)$

We keep the dominant term only.

2.2 Common Complexity Classes

Notation	Name	Example	n=10	n=100
$O(1)$	Constant	Array index access	1	1
$O(\log n)$	Logarithmic	Binary Search	3	7
$O(n)$	Linear	Linear Search	10	100
$O(n \log n)$	Linearithmic	Merge Sort	33	664
$O(n^2)$	Quadratic	Bubble Sort (worst)	100	10,000
$O(2^n)$	Exponential	Brute-force subsets	1024	$\sim 10^{30}$

Growth Rate (slowest to fastest growing):

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

GOOD PERFORMANCE <=====> BAD PERFORMANCE

Figure 2.1 – Big O growth hierarchy

2.3 Best, Worst, and Average Case

Every algorithm can be analysed under three scenarios depending on the input:

Three Cases of Algorithm Analysis

Best Case (Ω - O) = Minimum operations possible

e.g., searching and finding item at index 0

Average Case (Theta - O) = Expected operations over all inputs

e.g., item found in the middle on average

Worst Case (Big O) = Maximum operations possible

e.g., item not found -- must check every element

2.4 Calculating Big O – Step-by-Step Rules

- Drop constant multipliers: $O(5n) = O(n)$
- Drop lower-order terms: $O(n^2 + n) = O(n^2)$
- Nested loops multiply: $O(n)$ inside $O(n) = O(n^2)$
- Sequential loops add: $O(n)$ then $O(n) = O(n + n) = O(n)$
- Halving at each step: $O(\log n)$ (e.g., binary search)

```
// Example 1:  $O(n)$  - single loop
for (int i = 0; i < n; i++) { // runs n times
    System.out.println(i);    //  $O(1)$  work
} // Total:  $O(n)$ 

// Example 2:  $O(n^2)$  - nested loops
for (int i = 0; i < n; i++) { // outer: n times
    for (int j = 0; j < n; j++) { // inner: n times
        System.out.println(i+j); //  $O(1)$  work
    }
} // Total:  $O(n * n) = O(n^2)$ 

// Example 3:  $O(\log n)$  - halving
int i = n;
while (i > 1) {
    i = i / 2; // halves each time ->  $\log_2(n)$  steps
}
```

2.5 Space Complexity

Big O also applies to memory usage. Space complexity counts extra memory an algorithm allocates (not counting the input itself).

Algorithm	Time Complexity	Space Complexity
Linear Search	$O(n)$	$O(1)$ – no extra space
Binary Search	$O(\log n)$	$O(1)$ iterative / $O(\log n)$ recursive
Bubble Sort	$O(n^2)$	$O(1)$ – in-place
Merge Sort	$O(n \log n)$	$O(n)$ – needs auxiliary array

Recursive Fib	$O(2^n)$	$O(n)$ – call stack depth
---------------	----------	---------------------------

3. Recursion vs Iteration – Factorial & Fibonacci

3.1 What is Recursion?

Recursion is when a method calls itself to solve a smaller version of the same problem. Every recursive solution must have a BASE CASE (stopping condition) and a RECURSIVE CASE.

Two Laws of Recursion

1. BASE CASE -- A condition under which the function returns WITHOUT calling itself.

Without it, recursion never ends -> StackOverflowError!

2. RECURSIVE CASE -- The function calls itself with a SMALLER/SIMPLER input that moves toward the base case.

3.2 Factorial

The factorial of n (written $n!$) is the product of all positive integers from 1 to n .

$0! = 1$, $1! = 1$, $4! = 4 \times 3 \times 2 \times 1 = 24$

```
factorial(4) called
|-> 4 * factorial(3) [PUSH on call stack]
    |-> 3 * factorial(2) [PUSH]
        |-> 2 * factorial(1) [PUSH]
            |-> factorial(0) = 1 [BASE CASE]
                2 * 1 = 2 [POP]
                    3 * 2 = 6 [POP]
                        4 * 6 = 24 [POP]
                            Result: 24
```

Figure 3.1 – Call stack trace for factorial(4)

```
// RECURSIVE Factorial
public static int factorial(int n) {
    if (n == 0 || n == 1) return 1;    // BASE CASE
    return n * factorial(n - 1);       // RECURSIVE CASE
}

// Time: O(n)    Space: O(n) -- due to call stack

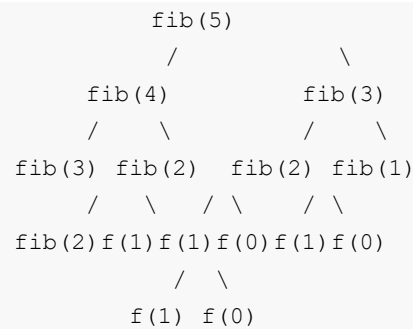
// ITERATIVE Factorial
public static int factorialIter(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

```
}
// Time: O(n)    Space: O(1) -- no call stack used
```

3.3 Fibonacci Sequence

The Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Each number is the sum of the two preceding ones: $F(n) = F(n-1) + F(n-2)$



Notice: fib(3) computed TWICE, fib(2) computed THREE times!
This is why naive recursion = $O(2^n)$ (exponential!)

Figure 3.2 – Recursive call tree for fib(5) showing repeated work

```
// RECURSIVE Fibonacci -- O(2^n) time -- SLOW!
public static int fib(int n) {
    if (n <= 1) return n;           // BASE CASE: fib(0)=0, fib(1)=1
    return fib(n-1) + fib(n-2);     // RECURSIVE CASE
}

// ITERATIVE Fibonacci -- O(n) time, O(1) space -- FAST
public static int fibIter(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1, c;
    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}

// DYNAMIC PROGRAMMING Fibonacci -- O(n) time, O(n) space
public static int fibDP(int n) {
    int[] memo = new int[n + 1];
    memo[0] = 0; memo[1] = 1;
    for (int i = 2; i <= n; i++) memo[i] = memo[i-1] + memo[i-2];
    return memo[n];
}
```

```
}
```

Method	Time	Space	When to use
Recursive	$O(2^n)$	$O(n)$	Never for large n
Iterative	$O(n)$	$O(1)$	Best for Fibonacci
Dynamic Prog	$O(n)$	$O(n)$	When memoisation is needed

Property	Recursion	Iteration
Mechanism	Calls itself	Uses loop (for/while)
Memory	Call stack grows $O(n)$	Fixed $O(1)$ memory
Speed	Overhead per call	Generally faster
Risk	<code>StackOverflowError</code>	Infinite loop
Readability	Elegant for trees/math	Can be verbose
Best for	Trees, divide & conquer	Simple counters, sums

4. Searching Algorithms – Linear & Binary Search

4.1 Linear Search

Linear Search checks every element from left to right until it finds the target or reaches the end. Works on ANY array (sorted or unsorted).

Array: [5, 3, 8, 1, 9, 2] Search for: 9

Step 1: 5 == 9? No

Step 2: 3 == 9? No

Step 3: 8 == 9? No

Step 4: 1 == 9? No

Step 5: 9 == 9? YES -> return index 4

Figure 4.1 – Linear Search trace

```
public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) return i;    // found at index i
    }
    return -1;    // not found
}

// Best Case:  O(1)    -- target is at index 0
// Worst Case: O(n)    -- target is last or not found
// Average:     O(n/2) = O(n)
```

4.2 Binary Search

Binary Search repeatedly halves the search space. **REQUIRES** a SORTED array. Compares the target to the middle element and eliminates half the remaining elements each step.

Sorted Array: [1, 3, 5, 7, 9, 11, 13, 15] Search for: 7

Step 1: low=0, high=7, mid=3 -> arr[3]=7 FOUND!

--- Another example, Search for: 11 ---

Step 1: low=0, high=7, mid=3 -> arr[3]=7 < 11 -> go RIGHT

Step 2: low=4, high=7, mid=5 -> arr[5]=11 FOUND!

--- Search for: 4 (not in array) ---

Step 1: low=0, high=7, mid=3 -> arr[3]=7 > 4 -> go LEFT

Step 2: low=0, high=2, mid=1 -> arr[1]=3 < 4 -> go RIGHT

Step 3: low=2, high=2, mid=2 -> arr[2]=5 > 4 -> go LEFT

Step 4: low=2, high=1 -> low > high -> NOT FOUND (-1)

Figure 4.2 – Binary Search trace on sorted array

```
// ITERATIVE Binary Search
public static int binarySearch(int[] arr, int target) {
    int low = 0, high = arr.length - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target) return mid;          // found!
        else if (arr[mid] < target) low = mid + 1; // search right
        else high = mid - 1;                        // search left
    }
    return -1; // not found
}

// RECURSIVE Binary Search
public static int binarySearchRec(int[] arr, int target, int low, int high) {
    if (low > high) return -1; // base case: not found
    int mid = (low + high) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target) return binarySearchRec(arr, target, mid+1, high);
    return binarySearchRec(arr, target, low, mid-1);
}
```

Property	Linear Search	Binary Search
Requirement	None (any array)	Array must be SORTED
Best Case	$O(1)$	$O(1)$
Worst Case	$O(n)$	$O(\log n)$
Average Case	$O(n)$	$O(\log n)$
Space	$O(1)$	$O(1)$ iterative, $O(\log n)$ recursive
Best for	Small/unsorted data	Large sorted data

Why Binary Search is $O(\log n)$

Each step halves the remaining elements:

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$

After k steps: $n / 2^k = 1 \Rightarrow k = \log_2(n)$

Example: $n = 1024 \Rightarrow$ at most $\log_2(1024) = 10$ steps

Compare: Linear Search could take 1024 steps!

5. Bubble Sort – Best, Worst & Average Case

5.1 How Bubble Sort Works

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each full pass, the largest unsorted element 'bubbles up' to its correct position at the end.

Array: [5, 1, 4, 2, 8]

PASS 1:

```
[5,1,4,2,8] -> compare 5,1 -> swap -> [1,5,4,2,8]
[1,5,4,2,8] -> compare 5,4 -> swap -> [1,4,5,2,8]
[1,4,5,2,8] -> compare 5,2 -> swap -> [1,4,2,5,8]
[1,4,2,5,8] -> compare 5,8 -> ok -> [1,4,2,5,8]
** 8 is now in final position **
```

PASS 2:

```
[1,4,2,5,8] -> compare 1,4 -> ok -> [1,4,2,5,8]
[1,4,2,5,8] -> compare 4,2 -> swap -> [1,2,4,5,8]
[1,2,4,5,8] -> compare 4,5 -> ok -> [1,2,4,5,8]
** 5 is now in final position **
```

PASS 3:

```
[1,2,4,5,8] -> compare 1,2 -> ok
[1,2,4,5,8] -> compare 2,4 -> ok
** No swaps -- SORTED! (early termination) **
```

Final: [1, 2, 4, 5, 8]

Figure 5.1 – Full Bubble Sort trace for [5, 1, 4, 2, 8]

```
public static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {           // n-1 passes
        boolean swapped = false;
        for (int j = 0; j < n - i - 1; j++) {  // shrinking inner loop
            if (arr[j] > arr[j + 1]) {
                // swap
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        // Optimisation: stop early if no swaps occurred
        if (!swapped) break;
    }
}
```

5.2 Best, Worst and Average Case Analysis

Case	Input	Comparisons	Swaps	Big O
Best Case	Already sorted: [1,2,3,4,5]	$n-1$	0	$O(n)$
Average Case	Randomly ordered	$\sim n^2/4$	$\sim n^2/4$	$O(n^2)$
Worst Case	Reverse sorted: [5,4,3,2,1]	$n(n-1)/2$	$n(n-1)/2$	$O(n^2)$

Best Case Explanation ($O(n)$)

If the array is already sorted, the inner loop runs once.

swapped stays FALSE -> the outer loop breaks after 1 pass.

Only $n-1$ comparisons are made -> $O(n)$.

This requires the 'swapped' optimisation flag in the code!

Without it, Bubble Sort is ALWAYS $O(n^2)$.

Worst Case Explanation ($O(n^2)$)

Reverse sorted array: every adjacent pair is out of order.

Pass 1: $n-1$ comparisons and swaps

Pass 2: $n-2$ comparisons and swaps

...

Pass $n-1$: 1 comparison

Total = $(n-1)+(n-2)+\dots+1 = n(n-1)/2 = O(n^2)$

5.3 Bubble Sort Properties

Property	Value
Time Complexity (Best)	$O(n)$
Time Complexity (Average)	$O(n^2)$
Time Complexity (Worst)	$O(n^2)$
Space Complexity	$O(1)$ – in-place sort
Stable Sort?	YES – equal elements keep original order
Adaptive?	YES – faster on nearly sorted data
Use Case	Small datasets or detecting sorted arrays

6. 2D & Multidimensional Arrays

6.1 Review – 1D Arrays

A 1D array is a sequence of elements stored in contiguous memory, accessed by a single index.

```
int[] arr = {10, 20, 30, 40, 50};  
//           0   1   2   3   4   (indices)  
System.out.println(arr[2]); // Output: 30  
// Time to access any element: O(1)
```

6.2 2D Arrays

A 2D array is an array of arrays — a table with rows and columns. Accessed using two indices: [row][column].

```
int[][] matrix = new int[3][4]   (3 rows, 4 columns)
```

	Col 0	Col 1	Col 2	Col 3
Row 0	[10]	[20]	[30]	[40]
Row 1	[50]	[60]	[70]	[80]
Row 2	[90]	[100]	[110]	[120]

```
matrix[1][2] = 70   (row 1, column 2)  
matrix[2][0] = 90   (row 2, column 0)
```

Figure 6.1 – 2D Array structure (3 rows x 4 columns)

```
// Declaration and initialisation  
int[][] matrix = {  
    {10, 20, 30, 40},  
    {50, 60, 70, 80},  
    {90, 100, 110, 120}  
};  
  
// Access element at row 1, col 2  
System.out.println(matrix[1][2]); // Output: 70  
  
// Traverse all elements (O(rows * cols))  
for (int i = 0; i < matrix.length; i++) { // rows  
    for (int j = 0; j < matrix[i].length; j++) { // cols  
        System.out.print(matrix[i][j] + " ");  
    }  
    System.out.println();  
}
```

6.3 Matrix Multiplication

To multiply matrix A ($n \times m$) by matrix B ($m \times p$), the result C is an $n \times p$ matrix. The number of columns in A must equal the number of rows in B.

$$\begin{array}{ccc} \text{A (2x3):} & & \text{B (3x2):} \\ \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} & \times & \begin{bmatrix} 7 & 8 \\ 9 & 10 \end{bmatrix} \\ & = & \begin{bmatrix} 1*7+2*9+3*11 & 1*8+2*10+3*12 \\ 4*7+5*9+6*11 & 4*8+5*10+6*12 \end{bmatrix} \\ & & \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix} \end{array}$$

$$\begin{aligned} C[0][0] &= 1*7 + 2*9 + 3*11 = 7+18+33 = 58 \\ C[0][1] &= 1*8 + 2*10 + 3*12 = 8+20+36 = 64 \\ C[1][0] &= 4*7 + 5*9 + 6*11 = 28+45+66 = 139 \\ C[1][1] &= 4*8 + 5*10 + 6*12 = 32+50+72 = 154 \end{aligned}$$

Figure 6.2 – Matrix multiplication example

```
// Matrix multiplication: O(n^3) time complexity
public static int[][] multiply(int[][] A, int[][] B) {
    int n = A.length;        // rows of A
    int m = B.length;        // rows of B (= cols of A)
    int p = B[0].length;     // cols of B
    int[][] C = new int[n][p];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < p; j++) {
            for (int k = 0; k < m; k++) {
                C[i][j] += A[i][k] * B[k][j]; // 3 nested loops = O(n^3)
            }
        }
    }
    return C;
}
```

6.4 Limitations of Arrays

- Fixed size: declared size cannot change at runtime
- Insertion/deletion is expensive: elements must shift — $O(n)$
- Wasted memory: if array is not full, unused slots waste space
- No built-in flexibility: cannot grow or shrink dynamically

This is WHY we need Linked Lists

Arrays are fast for random access ($O(1)$) but slow for insertion and deletion in the middle ($O(n)$ shifting).

Linked Lists solve this: insertion/deletion at any known node = $O(1)$

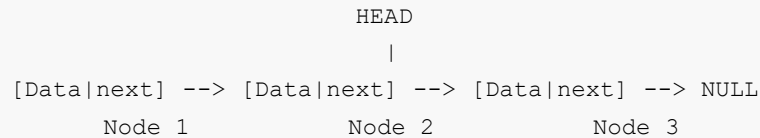
Trade-off: no random access -- must traverse from head = $O(n)$

7. Introduction to Linked Lists

7.1 What is a Linked List?

A Linked List is a linear data structure where elements (called nodes) are not stored in contiguous memory. Each node stores data AND a reference (pointer) to the next node.

Single Linked List:



HEAD = pointer to first node
NULL = marks the end of the list

Figure 7.1 – Single (Linear) Linked List structure

```
// Node class
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

// Linked List class
class LinkedList {
    Node head; // always points to first node

    // Insert at beginning -- O(1)
    public void insertFront(int data) {
        Node newNode = new Node(data);
        newNode.next = head;
        head = newNode;
    }

    // Insert at end -- O(n) to traverse
    public void insertEnd(int data) {
        Node newNode = new Node(data);
        if (head == null) { head = newNode; return; }
        Node current = head;
        while (current.next != null) current = current.next;
        current.next = newNode;
    }
}
```

```

}

// Delete node by value -- O(n)
public void delete(int data) {
    if (head == null) return;
    if (head.data == data) { head = head.next; return; }
    Node current = head;
    while (current.next != null && current.next.data != data)
        current = current.next;
    if (current.next != null) current.next = current.next.next;
}

// Traverse / print -- O(n)
public void display() {
    Node current = head;
    while (current != null) {
        System.out.print(current.data + " -> ");
        current = current.next;
    }
    System.out.println("NULL");
}
}

```

7.2 Array vs Single Linked List

Operation	Array	Linked List
Access by index	$O(1)$ – direct	$O(n)$ – traverse from head
Search	$O(n)$	$O(n)$
Insert at beginning	$O(n)$ – shift elements	$O(1)$
Insert at end	$O(1)$ amortised	$O(n)$ without tail pointer
Insert in middle	$O(n)$ – shift	$O(n)$ traverse + $O(1)$ link
Delete	$O(n)$ – shift	$O(n)$ traverse + $O(1)$ unlink
Size	Fixed at creation	Dynamic (grows/shrinks)
Memory	Contiguous, no overhead	Extra pointer per node

7.3 Limitation of Single Linked List

- Can only traverse in ONE direction (forward only)
- Cannot easily go backwards — must restart from head
- Deletion requires finding the PREVIOUS node — needs traversal
- No way to reach the previous node from a given node

This leads us to Doubly Linked Lists

To fix the one-directional limitation, each node gets TWO pointers:

-> next (points to next node)

-> prev (points to previous node)

This allows traversal in BOTH directions.

8. Doubly & Circular Linked Lists

8.1 Doubly Linked List

In a Doubly Linked List (DLL), each node has three parts: previous pointer, data, and next pointer. This allows traversal in both directions.

```
NULL <-- [prev|Data|next] <--> [prev|Data|next] <--> [prev|Data|next] --> NULL
           HEAD/Node 1           Node 2           Node 3
```

```
Each node: prev points LEFT, next points RIGHT
Head.prev = NULL, Tail.next = NULL
```

Figure 8.1 – Doubly Linked List structure

```
class DNode {
    int data;
    DNode prev, next;
    DNode(int data) { this.data = data; }
}

class DoublyLinkedList {
    DNode head, tail;

    // Insert at front -- O(1)
    public void insertFront(int data) {
        DNode newNode = new DNode(data);
        if (head == null) { head = tail = newNode; return; }
        newNode.next = head;
        head.prev = newNode;
        head = newNode;
    }

    // Insert at end -- O(1) with tail pointer
    public void insertEnd(int data) {
        DNode newNode = new DNode(data);
        if (tail == null) { head = tail = newNode; return; }
        tail.next = newNode;
        newNode.prev = tail;
        tail = newNode;
    }

    // Delete a node -- O(1) if node reference is given
    public void deleteNode(DNode node) {
        if (node.prev != null) node.prev.next = node.next;
        else head = node.next; // deleting head
        if (node.next != null) node.next.prev = node.prev;
    }
}
```

```

    else tail = node.prev; // deleting tail
}
}

```

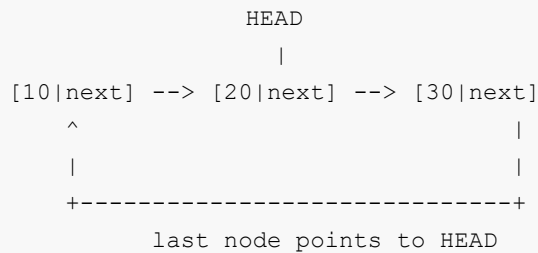
8.2 Advantages & Disadvantages of Doubly Linked List

Advantage	Disadvantage
Traverse in BOTH directions	Uses MORE memory (extra prev pointer per node)
Delete a node in $O(1)$ given ref	More complex insertion/deletion logic
Easier to insert before a node	Slightly slower due to two pointer updates
Tail pointer allows $O(1)$ insert at end	Still $O(n)$ for random access

8.3 Circular Linked List

In a Circular Linked List, the last node's next pointer points back to the first node (head) instead of NULL. This forms a circle — there is no end.

Single Circular Linked List:



No NULL -- the list wraps around!

Figure 8.2 – Circular Linked List

```

class CNode {
    int data;
    CNode next;
    CNode(int data) { this.data = data; }
}

class CircularLinkedList {
    CNode head = null;

    public void insertEnd(int data) {
        CNode newNode = new CNode(data);
        if (head == null) {
            head = newNode;

```

```

        head.next = head; // point to itself
        return;
    }
    // Find last node (last.next == head)
    CNode last = head;
    while (last.next != head) last = last.next;
    last.next    = newNode;
    newNode.next = head;    // wrap around
}

public void display() {
    if (head == null) return;
    CNode current = head;
    do {
        System.out.print(current.data + " -> ");
        current = current.next;
    } while (current != head); // stop when back at head
    System.out.println("(back to HEAD)");
}
}

```

8.4 Comparison of All Linked List Types

Feature	Single LL	Doubly LL	Circular LL
Direction	Forward only	Both directions	Forward (loops)
Prev pointer	No	Yes	No (or yes in CDLL)
Last node points to	NULL	NULL	HEAD
Memory per node	data + 1 ptr	data + 2 ptrs	data + 1 ptr
Use cases	Stacks, queues	Browser history, undo	Round-robin scheduling
Traversal end	NULL check	NULL check	Check if back at head

9. Quick Reference & Exam Tips

9.1 Big O Cheat Sheet

Operation	Data Structure	Complexity
Random access	Array	$O(1)$
Search (unsorted)	Array / List	$O(n)$
Search (sorted)	Array + Bin Srch	$O(\log n)$
Insert at front	Linked List	$O(1)$
Insert at end	Array (amort.)	$O(1)$
Insert/Delete (middle)	Array	$O(n)$
Insert/Delete (middle)	Linked List	$O(n)$ traverse + $O(1)$
Bubble Sort worst	Array	$O(n^2)$
Factorial (recursive)	--	$O(n)$ time, $O(n)$ space
Fibonacci (recursive)	--	$O(2^n)$ time -- SLOW
Fibonacci (iterative)	--	$O(n)$ time, $O(1)$ space

9.2 Key Formulas to Remember

- Bubble Sort total comparisons (worst): $n(n-1)/2$
- Binary Search steps: at most $\text{floor}(\log_2(n)) + 1$
- Factorial: $n! = n \times (n-1) \times (n-2) \times \dots \times 1$, $0! = 1$
- Fibonacci: $F(0)=0$, $F(1)=1$, $F(n) = F(n-1) + F(n-2)$
- Matrix multiply result size: $(n \times m) * (m \times p) = (n \times p)$

9.3 Linked List Pointer Rules

Critical Pointer Rules for Linked List Operations

ALWAYS save next pointer BEFORE changing it:

```
Node temp = current.next; // save
```

```
current.next = newNode; // change
```

Insert at front: `newNode.next = head;` then `head = newNode;`

Insert at end: traverse to last node, then `last.next = newNode;`

Delete head: `head = head.next;`

Circular check: `while (current.next != head) -- not != null`

9.4 Recursion Checklist

Before Writing Any Recursive Method

- ☐ What is the BASE CASE? (when does it stop?)
- ☐ What is the RECURSIVE CASE? (how does it call itself?)
- ☐ Is the input moving TOWARD the base case each call?
- ☐ Will this cause a StackOverflowError for large inputs?
- ☐ Would iteration be more efficient here?

9.5 Term 1 Exam Checklist

Topics You Must Be Comfortable With

- ☐ Define an algorithm and its 5 properties
- ☐ Explain and apply Big O notation (drop constants, keep dominant term)
- ☐ Identify best/worst/average case for a given algorithm
- ☐ Trace recursive factorial and Fibonacci by hand
- ☐ Compare recursion vs iteration (time, space, risk)
- ☐ Trace Linear Search and Binary Search step by step
- ☐ Explain WHY binary search requires a sorted array
- ☐ Perform and trace Bubble Sort (all cases) by hand
- ☐ Explain the 'swapped' flag and how it achieves $O(n)$ best case
- ☐ Access 2D arrays using double index [row][col]
- ☐ Trace matrix multiplication with small matrices
- ☐ List 4 limitations of arrays
- ☐ Implement insertFront, insertEnd, delete for a Singly Linked List
- ☐ Explain the difference between Singly, Doubly and Circular Linked Lists
- ☐ Draw diagrams for each linked list type